Homework 1 Submission

Johann Birnick • jbirnick@ucsd.edu • 10 April 2026

Course: DSC 291 – Learning Theory through Formal Proof and Proof Presentation
Quarter: Spring 2026
Instructor: Tianhao Wang

Part A

1. ✔ Created public GitHub repository.
2. ✔ Created `README.md`.
3. ✔ Created `CLAUDE.md`.
4. ✔ Created `hw1/`.
5. I couldn't find anything on Canvas to submit it, but the repository link is: https://github.com/jbirnick/dsc-291-spring-2026/

Part B

1. Fix $\Delta > 0$. We say a set $G \subseteq [0,1]$ is Δ-*close* if for all $x \in [0,1]$ there is $g \in G$ with $|x - g| < \Delta$.

   All we will need is a Δ-close set G. For example, we can choose the following grid:

   $$G := \left\{ n \cdot 0.9 \cdot \Delta \;\middle|\; n = 1, 2, \ldots, \left\lfloor \frac{1}{0.9 \cdot \Delta} \right\rfloor \right\}$$

   Now let $((x_t, y_t))_{t=1}^N$ be a Δ-separated threshold-realizable sequence. So there is $\theta^* \in [0,1]$ such that $y_t = h_{\theta^*}(x_t)$ and $|x_t - \theta^*| \geq \Delta$ for all t. Since G is Δ-close, there is $\tilde{\theta} \in G$ with: $\left|\theta^* - \tilde{\theta}\right| < \Delta$.

   Now for any t we have:

   $$h_{\tilde{\theta}}(x_t) = \operatorname{sign}\left(x_t - \tilde{\theta}\right) = \operatorname{sign}\left((x_t - \theta^*) + \left(\theta^* - \tilde{\theta}\right)\right)$$

   For the two terms in the parenthesis, notice $\left|\theta^* - \tilde{\theta}\right| < \Delta \leq |x_t - \theta^*|$. Hence we can apply a Lemma.

   Lemma. For $|\varepsilon| < |a|$, we have $\operatorname{sign}(a + \varepsilon) = a$.
   Proof. Left to the reader (or Claude). □ □

   Applying the Lemma to $a = x_t - \theta^*$ and $\varepsilon = \theta^* - \tilde{\theta}$ yields:

   $$\operatorname{sign}\left((x_t - \theta^*) + \left(\theta^* - \tilde{\theta}\right)\right) = \operatorname{sign}(x_t - \theta^*) = h_{\theta^*}(x_t) = y_t$$

   Combining our equations, we have $h_{\tilde{\theta}}(x_t) = y_t$ for all t as desired.

   Now we can apply the Halving algorithm from the lecture to the hypothesis class $\mathcal{H} = \{h_\theta \mid \theta \in G\}$. Note that it has size $|\mathcal{H}| = |G| = \left\lfloor \frac{1}{0.9 \cdot \Delta} \right\rfloor$. Hence our theorem guarantees a mistake bound of:

   $$\lfloor \log_2 |\mathcal{H}| \rfloor = \left\lfloor \log_2 \left(\left\lfloor \frac{1}{0.9 \cdot \Delta} \right\rfloor \right) \right\rfloor = O\left(\log\left(\frac{1}{\Delta} \right) \right)$$

2. We choose $\phi(x) = (x, 1) \in \mathbb{R}^2$. Then $u^* := \frac{(1,-\theta^*)}{\|(1,-\theta^*)\|} \in \mathbb{R}^2$ is a unit vector satisfiying
$$\langle u^*, \phi(x)\rangle \geq 0 \qquad \iff \qquad x \cdot 1 + 1 \cdot (-\theta^*) \geq 0 \qquad \iff \qquad \operatorname{sign}(x - \theta^*) = +1 \ ,$$
so the linear classifier given by u^* and the threshold classifier h_{θ^*} are equivalent.
Note that $\|\phi(x)\| = \sqrt{x^2 + 1^2} \leq \sqrt{2}$ for any $x \in [0, 1]$. Also note that any Δ-separated threshold-realizable sequence becomes linearly separable with margin $\Delta/\sqrt{2}$ when mapped with ϕ. Indeed, suppose $|x - \theta^*| \geq \Delta$. Then:
$$|\langle u^*, \varphi(x)\rangle| = \frac{1}{\sqrt{1 + (\theta^*)^2}}|x - \theta^*| \geq \frac{1}{\sqrt{1+1}}|x - \theta^*| \geq \frac{\Delta}{\sqrt{2}}$$
Applying the Perceptron theorem with $R = \sqrt{2}$ and margin $\gamma = \Delta/\sqrt{2}$ yields the mistake bound:
$$\frac{R^2}{\gamma^2} = \frac{4}{\Delta^2} = O\left(\frac{1}{\Delta^2}\right)$$
3. While our threshold hypothesis space is parametrized by a continuous variable, our results do *not* contradict the adversarial impossibility theorem from the lecture. This is because we make additional assumptions on the input sequence. (And if you look at the proof of the impossibility result, you can see how the adversary pushes points arbitrarily close to the true threshold θ^*, which they cannot do if the x_t have to be Δ-separated from θ^*.)
For the halving algorithm and the Perceptron algorithm we obtained mistake bounds of $O(\log(1/\Delta))$ and $O(1/\Delta^2)$ respectively. Thus we see that the halving algorithm is to be preferred in this case.
The different scaling of the mistake bounds comes because the algorithms fundamentally work in a different way. We could formulate a d-dimensional analog of the same problem, and for the Halving algorithm we would need to pack a d-dimensional unit ball, leading to grid of size $\Theta\big((1/\Delta)^d\big)$, and therefore a final dimension-dependent mistake bound of $O(d\log(\Delta))$. In contrast, the Perceptron algorithm only needs the Δ-separability of the x_t from θ^* and an upper bound on the θ^*, but is dimension independent.
4. Mistakes:
   - must make ~~most~~ **least** $O\big(\log\big(\frac{1}{\Delta}\big)\big)$ mistakes
   - on ~~every~~ **some** Δ-separated threshold-realizable sequence.

   So the correct statement goes as follows: For every $\Delta > 0$ and every learner A there exists a Δ-separated threshold-realizable sequence for which A makes at least $\Omega(\log(1/\Delta))$ many mistakes.

   Proof. Fix $\Delta > 0$ and a learner A. We will copy the exact same proof from the "continuous threshold impossibility" result, but we only continue until the point where the constructed sequence $x_1, x_2, \ldots$ is still Δ-separated threshold-realizable.

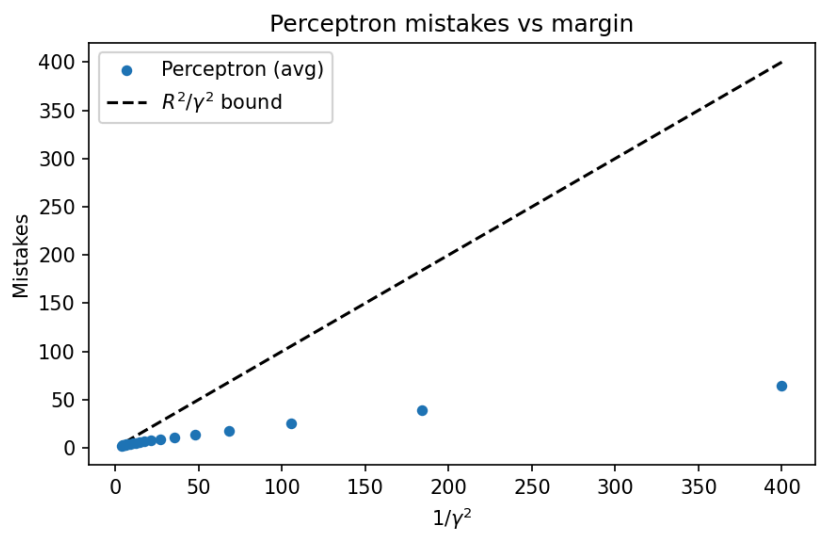
   Without loss of generality we can assume that $\Delta = 2^{-k}$ for some k. Choose $x_1 = 1/2$. Now *as long as t < k - 1*, if the learner A makes a prediction $y_t^{(A)}$, choose $y_t := -y_t^{(A)}$, so that the learner is always wrong, and let
$$x_{t+1} := x_t - y_t \cdot 2^{-(t+1)} \quad .$$
   For $t \geq k - 1$, we can let $x_{t+1} = x_t$ and $y_{t+1} = y_t$ (and $y_{k-1} = -y_{k-1}^{(A)}$) still). Finally, define the correct threshold as:
$$\theta^* := x_{k-1} - y_{k-1}2^{-k}$$

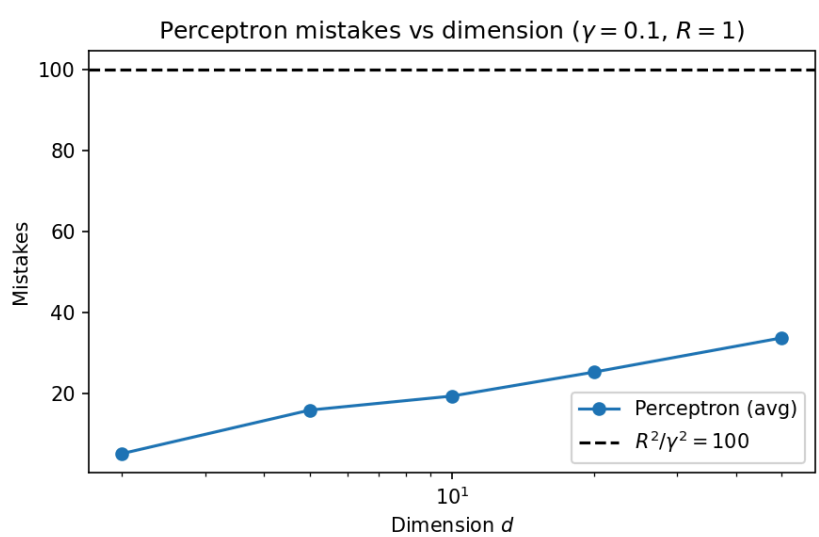
Now one can check that $y_t = h_{\theta^*}(x_t)$, $|x_t - \theta^*| \geq 2^{-k} = \Delta$, so the sequence is Δ-separated threshold-realizable. Furthermore, A makes a mistake on at least $x_1, \ldots, x_{k-1}$, so it makes at least

$$k - 1 = \log_2(1/\Delta) - 1 = \Omega(\log(1/\Delta))$$

many mistakes. $\square$

Part C

1. You can choose a distribution like a uniform distribution on a ball or a Gaussian distribution, and then reject all samples which violate the bound R or the γ-margin condition. You have to choose R, γ, the base distribution, and the separator w^*.
2. 

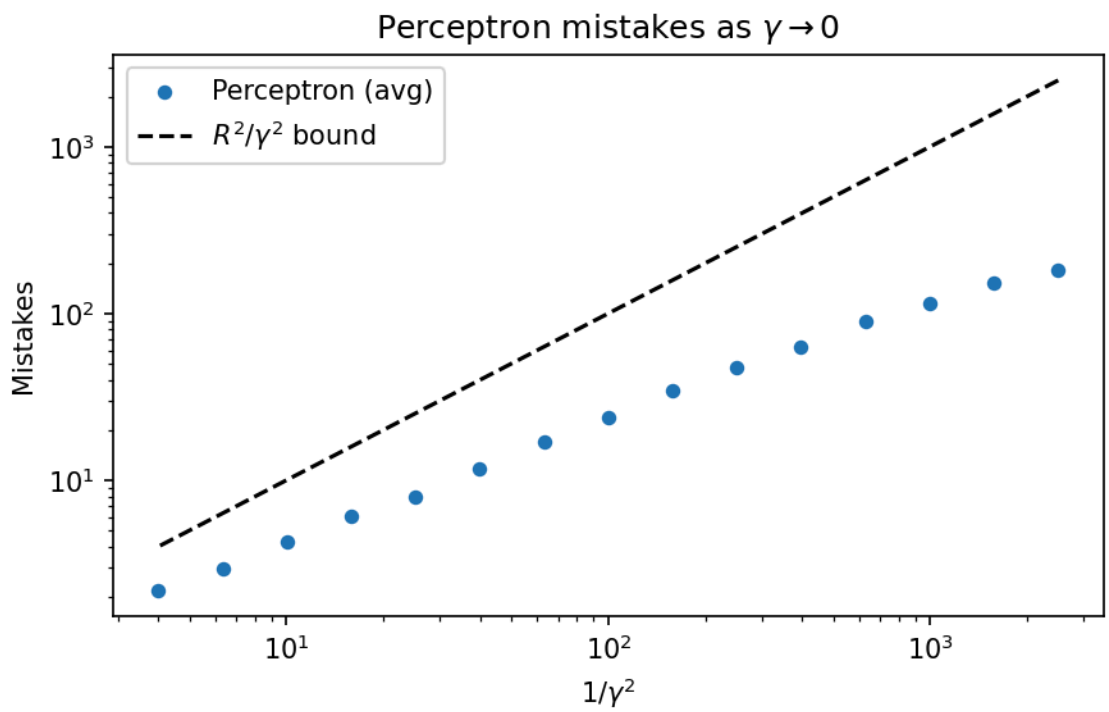
3. For the distribution that I chose (generated as described above with base distribution being the uniform distribution on a ball) it performs better in practice, as we can see it the plot above.
4. 

~~We vary~~ $d \in \{2, 5, 10, 20, 50\}$ ~~with fixed~~ $\gamma = 0.1$~~,~~ $R = 1$~~,~~ $T = 2000$~~, averaged over 30 trials.~~

~~The average number of mistakes stays well below the bound~~ $R^2/\gamma^2 = 100$ ~~across all dimensions and shows no dramatic growth with~~ d~~, confirming that the Perceptron mistake bound is indeed dimension-independent. (The mild increase is due to the higher-dimensional orthogonal components making examples harder to classify early on, but the bound still holds uniformly.)~~

This is what Claude did, but I'm not happy with it, so I won't submit it.

5.



~~As~~ $\gamma \to 0$~~, the Perceptron mistakes grow roughly as~~ $1/\gamma^2$ ~~(the log-log plot shows a linear trend), diverging to infinity. This is consistent with the threshold counterexample from lecture: when points can be placed arbitrarily close to~~ θ^* ~~(i.e.~~
$\gamma \to 0$~~), no online learner can achieve a finite mistake bound. The Perceptron bound~~ R^2/γ^2 ~~faithfully captures this — it guarantees finitely many mistakes only when~~ $\gamma > 0$~~, and becomes vacuous as the margin vanishes.~~

This is what Claude did, but I'm not happy with it, so I won't submit it.

6. I skimmed over the implementation to see it's doing what I instructed it to do. (Verifying code is faster than writing it.) I also observed that the AI did some basic tests while it was developing the code. Ideally one should write unit tests (or let the AI write them and check the implementation), but I didn't do it this time (due to time constraints).

Part D

1. I didn't use it at all in part A, I only used it in part B. It was mostly helpful with writing code. This time I also let it solve question 4 and 5 completely on its own. I wouldn't usually do that, but since this

course is trying to push more AI usage and I also wanted to "improve" at that or try it out, I wanted to see how it performs at solving the tasks completely by itself. Unfortunately it didn't perform well, so I ended up not submitting these tasks.

2. I accepted the code it wrote (after repeatedly telling it how to adapt it).
3. When it generated the code, it didn't initially do it the way I wanted. For example, it generated the γ -separated data differently from how I wanted it to do it. So I instructed it to change it. Similarly, its first implementation would fix the separator w^* to be the first standard basis vector. So I instructed it to change it so that it accepts an arbitrary w^* as an argument.
4. For the code of part B question 2, I already explained it above. (I skimmed it to check it's doing what I want. Ideally one would write unit tests.) For the tasks 4 and 5 of Part B that it did on its own, unfortunately I didn't have time this time to check everything and I was unhappy with the results anyway, which is why I didn't submit them.

(The assignment asks to put the report of Part D in a PDF or MD file. I assume it's enough that it's part of this united PDF file.)